

AutoSim Project – Phase Progress Report

Project Title:

AutoSim – A Hybrid Neuro-Symbolic Framework for Reliable Differential Equation Solving

Team Members:

- Devam Solanki
 - Kunjal Makwana
-

1. Project Objective

The primary objective of the AutoSim project is to design and implement a domain-specific intelligent system capable of solving Ordinary Differential Equations (ODEs) reliably and transparently.

Unlike general-purpose LLMs (e.g., ChatGPT, Gemini, etc.), which may hallucinate or generate incorrect symbolic manipulations for complex equations, AutoSim aims to:

- Classify differential equations correctly
- Decide between symbolic and numerical solution strategies
- Use verified mathematical solvers (SymPy / SciPy)
- Generate plots and numerical summaries
- Provide structured reasoning and step-by-step explanations

The system follows a hybrid approach combining:

- Rule-based mathematical logic
 - Verified symbolic computation
 - Verified numerical solvers
 - LLM-assisted reasoning (controlled usage)
-

2. Motivation

Large Language Models often:

- Hallucinate symbolic algebra
- Provide incorrect intermediate steps
- Generate mathematically invalid derivations

- Fail in stiff or nonlinear systems

AutoSim addresses this by ensuring that:

- Mathematical computation is performed by verified solvers
 - The LLM does not directly compute algebra
 - Solver routing is deterministic and rule-based
 - All outputs are mathematically grounded
-

3. Work Completed So Far

The following major components have been successfully implemented:

3.1 ODE Dataset Construction (Phase 1)

A structured dataset of differential equations was created covering:

- Linear ODEs
- Nonlinear ODEs
- Stiff systems
- Standard families (Duffing, Van der Pol, Logistic, etc.)

Each dataset entry includes:

- Equation
- Parameters
- Initial conditions
- Classification metadata
- Solver recommendations

This dataset is used for controlled LLM fine-tuning.

3.2 QLoRA Fine-Tuning Pipeline (Phase 2 – Stage A)

A fine-tuning pipeline was built using:

- 4-bit QLoRA

- Mistral-7B-Instruct
- Low VRAM optimization (8GB GPU environment)

Training setup included:

- Custom train/validation split
- Instruction-based formatting
- Controlled output design (classification-only behavior)
- Gradient checkpointing for memory efficiency

The model was trained to:

- Classify ODE type
- Detect linearity and stiffness
- Decide symbolic feasibility
- Recommend appropriate solver
- Avoid performing actual computation

3.3 Deterministic Rule-Based Solver Router

A rule-based routing system was implemented to:

- Detect equation order
- Detect nonlinearity
- Decide symbolic feasibility
- Select solver type (symbolic vs numerical)
- Identify equation families (Duffing, Van der Pol)

This ensures mathematical correctness independent of LLM output.

3.4 Symbolic Solver Integration

Implemented symbolic solving using:

- SymPy's dsolve
- Characteristic equation approach

- Support for:
 - Distinct real roots
 - Repeated roots
 - Complex roots

Example solved:

$$x'' + x = 0$$

Output:

$$x(t) = C_1 \sin(t) + C_2 \cos(t)$$

3.5 Numerical Solver Integration

Implemented numerical solving using:

- SciPy's solve_ivp
- RK45 integration method
- Automatic conversion of second-order ODE to first-order system
- Time-span based simulation
- Plot generation

Example solved:

$$x'' + 0.2x' + x + 0.1x^3 = \cos(t)$$

System outputs:

- Final value $x(t_f)$
- Maximum and minimum over interval
- Plot of $x(t)$

3.6 Hybrid Solve Pipeline

A complete hybrid pipeline was implemented:

Equation Input



Rule-Based Router



Symbolic Solver (if possible)

OR

Numerical Solver (if required)



Plot + Structured Summary



Step-by-Step Explanation

This ensures:

- No hallucinated algebra
 - Verified mathematical computation
 - Reproducible results
-

4. Current System Capabilities

The system can now:

- Solve second-order linear ODEs symbolically
 - Solve nonlinear ODEs numerically
 - Handle Duffing and Van der Pol oscillators
 - Generate plots
 - Provide structured reasoning
 - Operate independently of LLM hallucination
-

5. Individual Contributions

Partner Contribution

- Conducted extensive research on:
 - Neuro-symbolic AI approaches ◦ LLM hallucination in mathematical reasoning ◦ Hybrid solver architectures ◦ Differential equation solution strategies

- Identified solution pathways and research directions
- Evaluated existing literature and methodologies
- Proposed architectural design concepts for hybrid reasoning

My Contribution

- Designed and implemented the full system architecture
- Built and structured the AutoSim dataset
- Implemented QLoRA fine-tuning pipeline
- Developed deterministic solver router
- Integrated symbolic solver (SymPy)
- Integrated numerical solver (SciPy RK45)
- Implemented equation-family registry
- Built hybrid solve pipeline
- Implemented plotting and summary modules
- Performed end-to-end testing and validation
- Ensured reproducibility and mathematical correctness

All system-level implementation, training pipeline construction, solver integration, and experimental validation were carried out by me.

6. Current Stage

The project is currently at:

Hybrid ODE Solving Stage (Phase 2 Implementation Complete)

We have achieved:

- Reliable symbolic solving
- Reliable numerical solving
- Verified routing mechanism
- End-to-end execution pipeline

7 · Frontend Design Philosophy

The frontend of AutoSim was designed not merely as a user interface, but as an **interactive scientific computing environment**. The goal was to move beyond traditional form-based interfaces (e.g., Streamlit) and create a system that resembles modern computational tools such as MATLAB Online, Wolfram Alpha, and Desmos.

The design follows a structured workflow:

Understand → Define → Compute → Visualize → Interpret

Each section of the UI corresponds to a stage in this pipeline, ensuring that the user is guided logically through the process of solving differential equations.

Key principles followed:

- **Clarity:** Minimal cognitive load while inputting equations
- **Interactivity:** Immediate visual feedback via dynamic plots
- **Transparency:** Display of solver decisions and computation steps
- **Trust:** Emphasis on "Zero Hallucination" — all numerical results are deterministic

8 • Frontend–Backend Integration

The frontend communicates with the AutoSim backend via a REST API layer. The integration ensures a clear separation between:

- **Computation Layer (Python)**
- **Visualization & Interaction Layer (Next.js)**

Workflow:

1. User inputs equation and parameters
2. Frontend sends request to backend API
3. Backend:
 - Parses equation
 - Routes to appropriate solver
 - Computes solution
4. Frontend:
 - Receives structured response
 - Renders plots and insights
 - Displays LLM-generated explanations

This architecture ensures:

- modularity
- scalability
- maintainability

9 • Visualization Strategy

Instead of relying on standard plotting libraries, the visualization panel was designed using a **custom Canvas 2D rendering system**.

This approach provides:

- higher rendering performance
- greater control over styling
- smooth animations and transitions
- custom visual elements such as:

- neon glow effects
- phase space arrows
- gradient-filled trajectories

Two primary visualizations are implemented:

- **Time-series plot ($x(t)$)**
- **Phase portrait (x vs x')**

The ability to toggle between views allows users to analyze both **temporal behavior** and **dynamical system properties**.

10 · User Experience (UX) Design

The UI was intentionally structured as a **single-page scrollable experience**, allowing users to move seamlessly between:

- introduction
- solver workspace
- results dashboard

Key UX features:

- **Progressive disclosure** (simple → advanced information)
- **Tabbed result exploration**
- **Interactive solver selection**
- **Live feedback during computation**

Special attention was given to **visual hierarchy**, ensuring that:

- inputs are clearly separated from outputs
- results are grouped logically
- important metrics are highlighted

11 · Role of LLM Integration in UI

The frontend integrates LLM outputs as an **explanatory layer**, not as a computational engine.

Displayed outputs include:

- natural language explanation of equation
- physical interpretation
- classification of system

This improves:

- accessibility for beginners
- interpretability of results
- educational value of the platform

12 · Collaborative Development

The frontend development was carried out collaboratively.

- **Primary System Design & Backend Integration:** Devam P. Solanki
- **Frontend UI Development & Component Implementation:** [Kunjai Makwana]

The frontend implementation, including:

- layout structuring
- component design

- styling system
- interactive panels

was significantly contributed by the collaborator.

This collaborative approach enabled:

- faster development cycles
- separation of concerns (backend vs frontend)
- improved UI quality

13 • Advantages Over Previous Streamlit Interface

The previous Streamlit-based interface was limited in:

- layout flexibility
- visual design
- interactivity
- scalability

The new frontend provides:

Feature	Streamlit New Frontend	
UI Flexibility	Low	High
Interactivity	Limited	Advanced
Visualization	Basic	Custom Canvas
User Experience	Linear	Interactive
Scalability	Limited	High

14 • Future Enhancements

The frontend architecture allows several future improvements:

- Multi-method comparison (Euler vs RK4 vs RK45)
- 3D PDE visualization
- Real-time simulation controls
- Export of results (CSV, plots)
- Integration with reinforcement learning-based solver selection

15 • Conclusion

The AutoSim frontend transforms the system from a simple solver into an **interactive computational laboratory for differential equations**.

By combining:

- modern web technologies
- scientific visualization
- deterministic computation
- AI-assisted explanation

the platform achieves both:

- **engineering robustness**
- **educational usability**

This makes AutoSim suitable for:

- academic learning
- research experimentation
- engineering analysis